

M2 Data Science – M2 Intelligence Artificielle

Data Mining

Repérer des patterns
Détecter des anomalies



Comment repérer des comportements récurrents ?



Questions:

- Quelles cartes sont souvent utilisées ensemble ?
- Quelles couleurs sont régulièrement combinées ?
- Quelles cartes sont plutôt utilisées en début ou fin de partie ?
- Quels enchainements de cartes sont récurrent ?

Repérer des patterns

Place à la pratique !

- Sur [kaggle](#), téléchargez l'un des jeux de données suivants:
 1. [Magic: The Gathering - Winning Pioneer Decks](#)
 2. [Market Basket Analysis](#)
 3. [League of Legends Ranked Matches](#)
 4. [European Soccer Database](#)
 5. [Pokemon TCG - All Tournaments Decks](#)
 6. [Chess Games](#)
 7. [Brewer's Friend Beer Recipes](#)
- Vérifiez et nettoyez vos données !
 - Ex. Formats des dates, encodage des données, etc.
- Discutez sur quelles variables peuvent cacher des patterns et sur comment les repérer
- Faites des tests et discutez de vos résultats



Repérer des patterns

Matrice de co-occurences

					
	15	5	3	7	0
	5	20	6	8	3
	3	6	12	2	8
	7	8	2	14	9
	0	3	8	9	17

Bonne pratique:

⇒ Mettre sous forme de pourcentage par ligne ou colonne

Limite:

⇒ Fonctionne moins bien sur autre chose que des pairs

Repérer des patterns

Gérer les combinaisons de plus de deux éléments

Définitions:

Items: $I = \{i_1, i_2, i_3, \dots\}$

- I : ensemble de tous les éléments/item possible

Transaction: t

- Tel que t est un sous ensemble de I

Database: $D = \{t_1, t_2, t_3, \dots\}$

- D : ensemble de transactions

Itemset: X

- Tel que X est un sous ensemble de I
- On cherche dans D les transactions contenant X

$$I = \{ \text{☀️}, \text{🌳}, \text{💧}, \text{🌀}, \text{💀} \}$$

transactionID	☀️	🌳	💧	🌀	💀
001	1	0	0	1	0
002	0	1	1	0	0
003	1	1	1	1	0
004	0	0	1	0	1
005	0	0	1	1	1
006	1	0	0	1	0
007	1	0	0	0	1

$$X = \{ \text{☀️}, \text{🌀} \}$$

Repérer des patterns

Évaluer la fréquence des itemsets

$I = \{ \text{☀️}, \text{🌳}, \text{💧}, \text{🌀}, \text{💀} \}$

transactionID	☀️	🌳	💧	🌀	💀
001	1	0	0	1	0
002	0	1	1	0	0
003	1	1	1	1	0
004	0	0	1	0	1
005	0	0	1	1	1
006	1	0	0	1	0
007	1	0	0	0	1

$X = \{ \text{☀️}, \text{🌀} \}$

Définitions:

Support absolu:

- Nombre de transactions dans **D** contenant **X**
- Dans notre exemple: 3

Support relatif (ou juste Support):

- Fraction des transactions dans **D** contenant **X**
- Dans notre exemple:

$$\text{abs_support}(X) / |D| = 3 / 7 = 0,43$$

- Donne aussi une estimation de **P(X)**, la probabilité pour une transaction de contenir **X**

Un itemset est considéré fréquent ssi:

- $\text{rel_support}(X) \geq \text{min_support}(I, D)$

Repérer des patterns

Trouver des associations d'itemsets

Définitions:

Association: $X \rightarrow Y$

- « Si X est dans une transaction alors Y aussi »
- Telle que:
 - $X \subseteq I, Y \subseteq I$
 - $X \cap Y = \emptyset$

Support d'une association $X \rightarrow Y$:

- Support de $X \cup Y$
- Dans notre exemple: 1 ou 1/7

On cherche des associations « intéressantes »

- Au risque de trouver des associations fallacieuses

$$I = \{ \text{☀️}, \text{🌳}, \text{💧}, \text{🌀}, \text{💀} \}$$

$$X = \{ \text{☀️}, \text{🌀} \}$$

$$Y = \{ \text{💧}, \text{🌳} \}$$

transactionID	☀️	🌳	💧	🌀	💀
001	1	0	0	1	0
002	0	1	1	0	0
003	1	1	1	1	0
004	0	0	1	0	1
005	0	0	1	1	1
006	1	0	0	1	0
007	1	0	0	0	1

Repérer des patterns

Évaluer l'intérêt d'une association

$I = \{ \text{☀️}, \text{🌳}, \text{💧}, \text{🐉}, \text{💀} \}$

$X = \{ \text{☀️}, \text{🐉} \}$

$Y = \{ \text{💧}, \text{🌳} \}$

transactionID	☀️	🌳	💧	🐉	💀
001	1	0	0	1	0
002	0	1	1	0	0
003	1	1	1	1	0
004	0	0	1	0	1
005	0	0	1	1	1
006	1	0	0	1	0
007	1	0	0	0	1

Confidence: $[0 .. 1]$

- $conf(X \rightarrow Y) = P(Y|X) = \frac{support(X \cap Y)}{support(X)}$
- Dans notre exemple: $0,14 / 0,43 = 0,32$

Lift: $[0 .. \infty[$

- $lift(X \rightarrow Y) = \frac{conf(X \rightarrow Y)}{support(Y)}$
- Dans notre exemple: $0,32 / 0,29 = 1,1$

Leverage: $[-1 .. 1]$

- $leverage(X \rightarrow Y) = support(X \rightarrow Y) - support(X) \times support(Y)$
- Dans notre exemple: $0,14 - 0,43 \times 0,29 = 0,02$

Et bien d'autres !

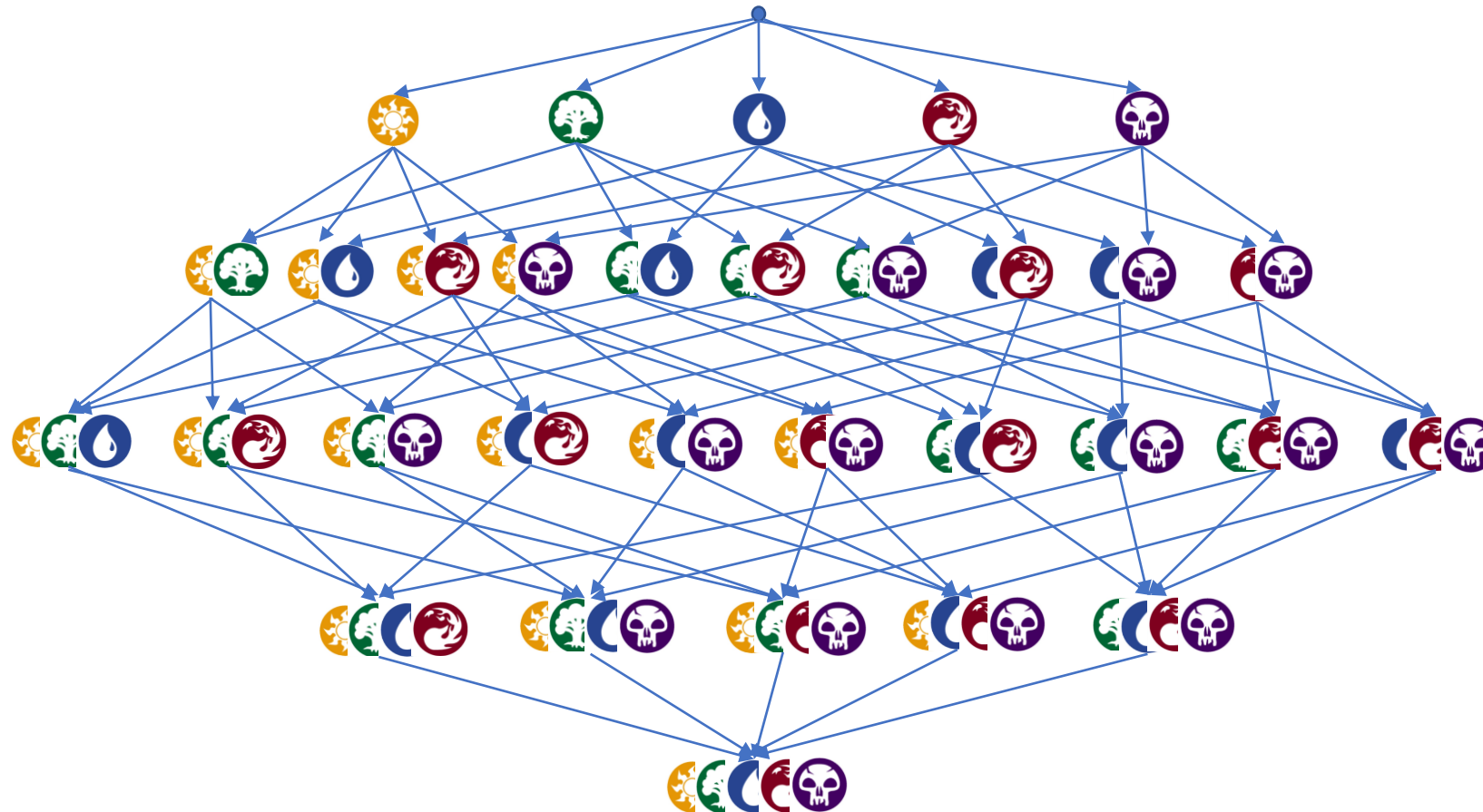
https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/

Repérer des patterns

Extraire des itemsets

Approche naïve

=> Calculer le support de tous les itemsets possibles



Problème:

- Avec 5 items -> 31 itemsets possibles
- Avec 1000 items -> 2^{1000} itemsets possibles

Solution possible:

- Le support est anti-monotone
 - Si X_1 est fréquent, alors le sous-ensemble X_2 de X_1 est fréquent
 - Si X_1 n'est pas fréquent, alors X_2 , tel que X_1 est un sous-ensemble de X_2 , n'est pas fréquent
- Chercher en priorité les itemsets dont les sous-ensembles sont fréquents

Repérer des patterns

L'algorithme APriori

Apriori_gen(L, k): L = itemsets fréquents à k-1 k = taille des itemsets recherchés

result <- list()

Pour chaque $p \in L$, $q \in L$ **tels que** $p_1 = q_1, p_2 = q_2, \dots, p_{k-2} = q_{k-2}$ **et** $p_{k-1} < q_{k-1}$: Génère des combinaisons unique sans répétitions

$c = p \cup \{q_{k-1}\}$

Si $u \in L$ **pour tout** $u \subseteq c$ **tels que** $|u| = k - 1$: Pruning: check aussi que tous les itemsets sont fréquents

 result.add(c)

Return Union(L_k)

APriori(T, ε):

L_1 <- {1-itemsets fréquents}

k <- 2

Tant que L_{k-1} **n'est pas vide:**

C_k <- Apriori_gen(L_{k-1} , k)

Pour chaque transaction t **dans** T:

D_t <- {c dans C_k : $c \subseteq t$ }

Pour chaque candidat c **dans** D_t :

 count[c] <- count[c] + 1

L_k <- {c dans C_k : count[c] ≥ ε}

 k <- k + 1

Return Union(L_k)

Commence par rechercher les itemsets de taille 2

← Génère des candidats

Calcule le support

Check si le support dépasse le seuil ε
Augmente la taille des itemsets recherchés

Limite:

- Scanne plusieurs fois le dataset

Alternative:

- FP-Growth
- ECLAT
- PrefixSpan
- Distributed approaches (Spark...)
- Et bien d'autres...

Repérer des patterns

Beaucoup d'autres axes de travail

Comment gérer les valeurs numériques ?

- Discrétiser les valeurs des variables concernées
- Repérer des intervalles pertinents
 - Ex. $\{a \in [0..10[, b \in [20..30[\}$
- Repérer des (sous-)répartitions (loi normale, probabilités, logique floue, etc.)

Comment prendre en compte l'ordre des évènements ? (Cf. Cours 4)

- Graph Mining
- Process Mining
- Sequential Pattern Mining

Comment prendre en compte la localisation des évènements ? (Cf. Cours 5)

- Graph Mining
- Ex. Différences d'achats entre les supermarchés proches des écoles et ceux proches de bureaux

Détecter des anomalies

Place à la pratique !

- Reprenez les jeux de données téléchargés précédemment:
 1. [Magic: The Gathering - Winning Pioneer Decks](#)
 2. [Market Basket Analysis](#)
 3. [League of Legends Ranked Matches](#)
 4. [European Soccer Database](#)
 5. [Pokemon TCG - All Tournaments Decks](#)
 6. [Chess Games](#)
 7. [Brewer's Friend Beer Recipes](#)
- Essayez de détecter des « anomalies » (et discutez de ce que peut être une « anomalie »)
- Faites des tests et discutez de vos résultats



Détecter des anomalies

Qu'est ce qu'une « anomalie » ?

«Évènement(s) ne correspondant(s) pas à un pattern connu»

Exemple: retraits bancaires d'un individu

Pattern connu:

- Retraits de 65.8 ± 25.9 \$ en moyenne
- Localisés à New-York et ses alentours

ID	Lieu	Montant
118	NY	60\$
119	Brooklyn	40\$
120	NY	120\$
121	NY	55\$
122	Moscou	70000\$
123	Brooklyn	70\$
124	NY	50\$

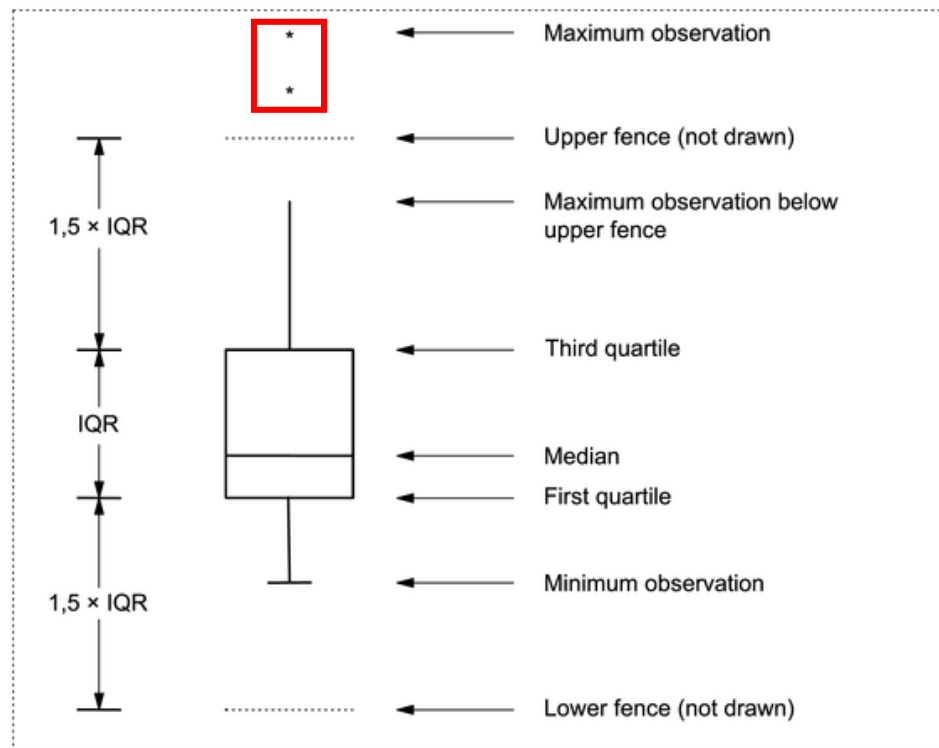
« Anomalie »:

- Retrait très au dessus de la moyenne habituelle
- Localisé à un endroit inhabituel

Détecter des anomalies

Détection univariée

Valeurs « aberrantes »
(outliers)



Intervalle interquartile (IQR):

- L'IQR est la distance entre le premier quartile Q1 et le troisième quartile Q3 d'un ensemble de valeurs
- Par convention, on fixe un seuil tel que si une valeur est inférieure à $Q1 - 1.5 \times IQR$ ou supérieure à $Q3 + 1.5 \times IQR$, alors:
 - La valeur est considérée comme « aberrante »

Z-score:

- Le nombre d'écart-types entre la valeur et la moyenne
 - $z = \frac{x - \bar{x}}{\sigma}$, avec $\bar{x}$ la moyenne et σ l'écart-type
 - Dans notre exemple: $z = \frac{70000 - 10\ 056}{24\ 472} = 2.45$
- Par convention, une valeur est considérée « aberrante » si son z-score est supérieur à 3

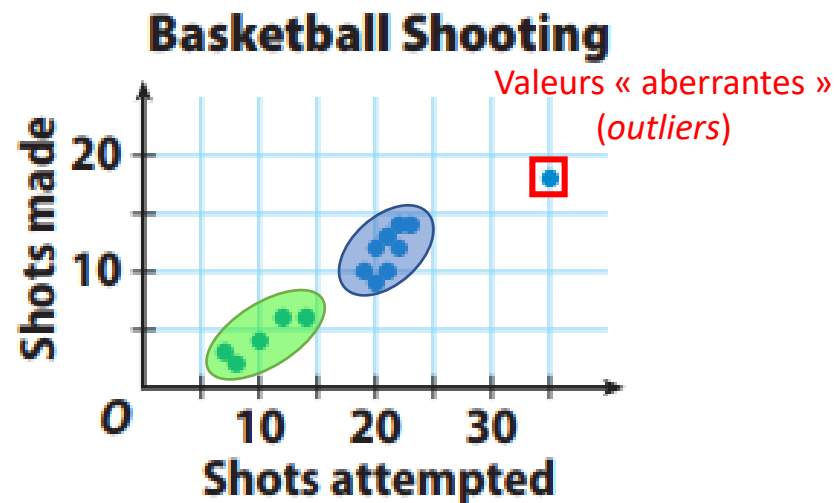
Z-score modifié:

- Se base sur la médiane et l'écart absolu médian (EAM) plutôt que sur la moyenne et l'écart-type
 - $z = \frac{x - \text{median}}{EAM}$
 - Dans notre exemple: $z = \frac{70000 - 60}{10006} = 6.99$

⇒ Reste à savoir si une valeur « aberrante » est une « anomalie »

Détecter des anomalies

Détection multivariée

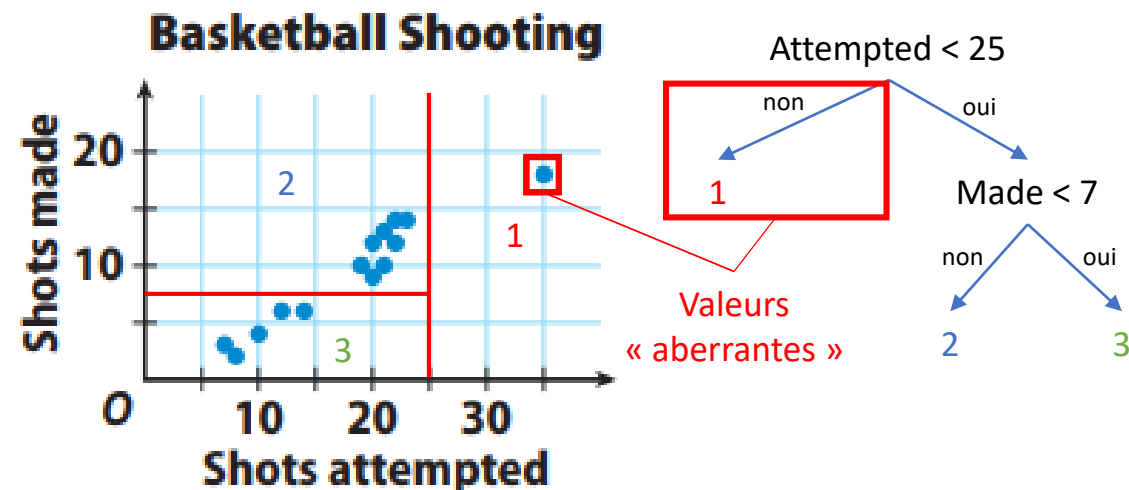


Approche par regroupement (*clustering*):

- On applique un algorithme de regroupement (k-means, DB-Scan, etc.)
 - De préférence DB-Scan
- Les valeurs n'ayant pas pu être groupé sont considérées « aberrantes »

Forêt d'isolement (*Isolation Forest*):

- On génère un ensemble d'arbre de décision en divisant l'ensemble des données récursivement
- Les valeurs isolées le plus rapidement sont considérées « aberrantes »



Détecter des anomalies

Angle-based Outlier Detection (ABOD)

ABOD(D, ϵ): # D un jeu de donnée et ϵ un seuil

```
anomalies_potentielles <- list()
```

Pour chaque donnée $d \in D$:

```
l <- list()
```

Pour chaque donnée $d' \in D, d' \neq d$:

```
a <- angle(d, d')
```

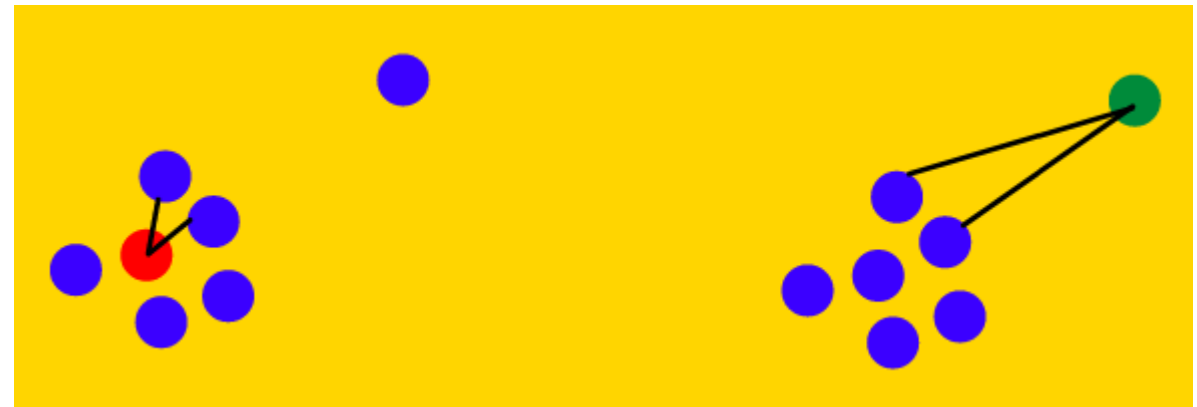
```
l.add(a)
```

```
v <- variance(l)
```

Si $v < \epsilon$:

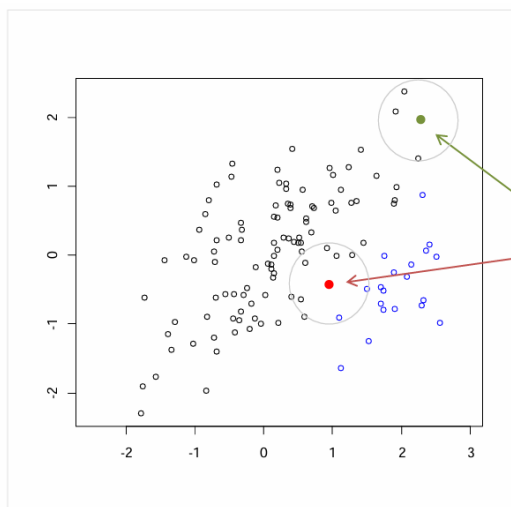
```
anomalies_potentielles.add(d)
```

Return anomalies_potentielles

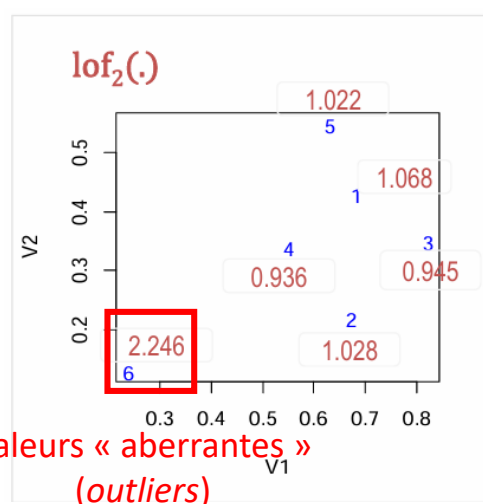
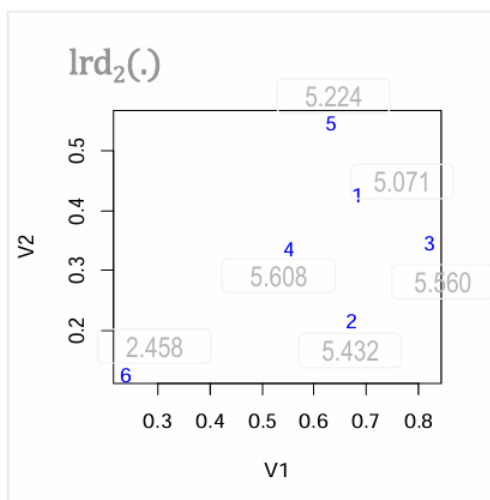


Détecter des anomalies

Local Outlier Factor (LOF)



Dans une zone à forte densité, un point qui s'écarte des autres (de ses voisins immédiats) devrait plus interroger que lorsqu'il se situe dans une zone moins dense.



k-distance(A):

- Distance entre A et son k-ème voisin le plus proche

Distance atteignable (*reachability distance*):

- $rd_k(A, B) = \max(kdistance(B), d(A, B))$
- Les points inclus entre B et son k-ème voisin sont considérés équivalents

Densité locale (*local reachability density*):

- $lrd_k(A) = \frac{1}{\frac{\sum_{B \in N_k(A)} rd_k(A, B)}{|N_k(A)|}}$, avec $N_k(A)$ le voisinage d'ordre k de A
- Inverse du rd_k moyen d'un point avec son voisinage d'ordre k

Facteur local d'anomalie (*local outlier factor*):

- $lof_k(A) = \frac{\sum_{B \in N_k(A)} \frac{lrd_k(B)}{lrd_k(A)}}{|N_k(A)|}$
 - LOF ≈ 1 , densité similaire à ses voisins
 - LOF < 1 , densité plus élevée que ses voisins (*inlier*)
 - LOF > 1 , densité plus faible que ses voisins (*outlier*)

Bonne pratique:

=> Trier les résultats pour identifier les valeurs suspectes (LOF très élevé)

Place à l'action !



Votre Mission:

- Reprendre votre code là vous l'aviez laissé
 - Lien initial: <https://github.com/a-t-richard/20262027-UE-DataMining-Project>
- Utiliser les nouvelles notions et méthodes vues pendant ce cours
- Trouver ce qu'il se cache dans ces données !